

BUILD-SUPABASE.md

Partie C.2 — Organisation des schémas (Schemas)

Version : V1

Statut : Référence officielle de l'organisation des schémas PostgreSQL

1. Objectif

Cette section définit les règles officielles d'organisation des schémas PostgreSQL dans Vitala.

Elle garantit :

- une séparation claire des responsabilités ;
- une meilleure sécurité ;
- une maintenance simplifiée ;
- une architecture évolutive.

Chaque objet PostgreSQL (table, vue, fonction, trigger...) doit appartenir à un schéma clairement identifié.

2. Principes directeurs

L'organisation des schémas repose sur les principes suivants :

- Séparation des responsabilités
- Domain First
- Sécurité par défaut
- Modularité
- Évolutivité
- Documentation obligatoire

Aucun schéma ne doit avoir une responsabilité ambiguë.

3. Schémas système

Les schémas gérés par Supabase sont réservés à l'infrastructure.

Ils comprennent notamment :

- `auth` : gestion des utilisateurs, sessions et authentification.
- `storage` : gestion des buckets et des fichiers.
- `extensions` : extensions PostgreSQL activées.
- `graphql_public` (si utilisé) : exposition GraphQL.

Ces schémas ne doivent pas être modifiés directement sauf lorsque la documentation officielle de Supabase le prévoit.

4. Schéma public

Le schéma `public` constitue le schéma principal des données métier.

Il contient :

- les tables des domaines ;
- les vues métier ;
- les fonctions SQL autorisées ;
- les politiques RLS ;
- les index.

Toutes les conventions définies dans BUILD-DATABASE.md et BUILD-DOMAINS.md s'y appliquent.

5. Organisation logique des domaines

Même si les tables résident dans le schéma `public`, elles sont organisées par domaine grâce à des conventions de nommage cohérentes.

Exemples :

- `udi_*`
- `flash_*`
- `mission_*`
- `trust_*`
- `media_*`
- `sync_*`
- `audit_*`
- `notification_*`

Cette convention facilite la lecture, la maintenance et les migrations.

6. Schémas techniques futurs

Si les besoins de Vitala évoluent, des schémas supplémentaires pourront être créés pour des usages spécifiques, par exemple :

- analytics
- archive
- integration
- reporting

La création d'un nouveau schéma doit être validée au niveau de l'architecture et documentée.

7. Fonctions SQL

Les fonctions SQL doivent être placées dans le schéma correspondant à leur responsabilité.

Les fonctions purement techniques peuvent être regroupées dans un schéma dédié si leur nombre devient important.

Les fonctions métier restent limitées afin de préserver le principe Domain-Driven.

8. Vues

Les vues doivent être créées dans le même schéma que les données qu'elles représentent, sauf justification contraire.

Les vues matérialisées doivent être clairement identifiées et documentées.

9. Triggers

Les triggers sont rattachés aux tables de leur domaine.

Ils doivent rester simples, documentés et limités à des besoins techniques tels que :

- audit ;
- mise à jour de métadonnées ;
- synchronisation technique.

Ils ne doivent pas implémenter des règles métier complexes.

10. Sécurité des schémas

Chaque schéma possède ses propres règles d'accès.

Les permissions doivent être attribuées selon le principe du moindre privilège.

Les schémas système restent protégés contre les modifications non autorisées.

11. Documentation

Chaque schéma doit disposer d'une documentation précisant :

- son objectif ;
 - les objets qu'il contient ;
 - les règles particulières qui lui sont applicables ;
 - les dépendances éventuelles.
-

12. Évolution

L'ajout d'un nouveau schéma doit :

- répondre à un besoin clairement identifié ;
 - éviter les duplications ;
 - préserver la compatibilité avec l'architecture existante ;
 - être accompagné des migrations et de la documentation correspondantes.
-

13. Critères de conformité

L'organisation des schémas est conforme lorsque :

- chaque objet appartient à un schéma clairement identifié ;
 - les responsabilités sont séparées ;
 - les conventions de nommage sont respectées ;
 - les accès sont sécurisés ;
 - la documentation est à jour.
-

14. Conclusion

L'organisation des schémas PostgreSQL constitue l'un des fondements de la maintenabilité de Vitala.

Une structure claire des schémas permet d'assurer une évolution maîtrisée, une meilleure sécurité et une compréhension immédiate de la base de données par les développeurs comme par les intelligences artificielles.